

项目难点

请介绍一下你的项目

你的回答：

(1. 业务全景 —— 简单大方)

“好的。这个项目是一个对标 B 站核心功能的分布式在线视频平台。

从业务上看，它主要包含两个端：**用户端**负责视频观看、弹幕互动和个人中心；**审核端**负责视频的审发和内容管理。

整个系统的核心链路就是：**用户投稿 -> 平台审核转码 -> 线上分发播放。**”

(2. 架构选型 —— 一笔带过微服务)

“在技术架构上，我采用了 **Spring Cloud Alibaba** 微服务体系。

之所以这么选型，不是为了拆分而拆分，主要是考虑到**业务属性的差异**：

比如‘视频转码’是 CPU 密集型的重任务，而‘点赞评论’是高并发 IO 的轻任务。

通过微服务将它们拆开，利用 **Nacos** 和 **Gateway** 进行治理，主要是为了实现**故障隔离**，防止后台转码把前台的登录服务拖垮。”

(点到为止，表明你懂微服务的核心目的就是解耦，然后立马转折)

(3. 抛出诱饵 —— 引导提问)

“虽然微服务搭建了系统的骨架，但在开发过程中，我发现**真正的挑战**其实在于那条核心的**视频处理流水线**。

因为在像抖音直播这种场景下，我们要处理 **GB 级别的大文件上传**，还要在**不稳定的网络**和**复杂的审核流程**中，保证视频数据不丢、状态不错乱。

这也是我在这个项目里投入精力最多、思考最深的地方。”

我觉得最难的点在于设计并实现了一个，高效且可靠高可靠视频处理流水线

因为在抖音直播这种场景下，视频处理不仅仅是把文件存下来那么简单，它面临三个维度的挑战：

第一是网络传输的不稳定性（大文件上传）；

第二是业务状态的复杂性（用户反复修改稿件）；

第三是计算资源的高消耗**（因为要把 MP4 格式的文件转换成流式播放的 m3u8 和 ts，FFmpeg 转码是 CPU 密集型任务）。

为了解决这些问题，我设计了一套**从预上传，分片上传到异步转码的完整方案。**”

第一阶段：解决传输难题 —— 预上传与分片

首先是上传阶段。为了解决大文件（GB 级别）上传容易失败的问题，我没有用传统的文件流，而是设计了**基于 Redis 的分片断点续传机制**。

具体来说，我把文件切分成多个分片。前端上传前，先通过‘预上传接口’在 Redis 里初始化一个状态机。上传过程中，后端利用 Redis 实时记录上传进度。

这里最难的处理在于**并发上传时的数据一致性**，以及**如何保证合并后的文件不损坏**。我通过在 Redis 层面做原子性校验，以及在 IO 层面做有序合并，确保了即使在网络极差的情况下，可靠传输和断点续传。”

第二阶段：解决业务难题 —— 读写分离与增量更新)

“文件传上来只是第一步，业务上最复杂的点在于**视频的审核与发布流程**。

为了保证用户在修改稿件（比如删减分片、改标题）时，不影响线上观众的观看体验，我设计了‘**4表架构**’，即审核态（Draft）与发布态（Publish）完全物理隔离。

在处理用户修改时，为了避免全量重新转码浪费资源，我设计了一套‘**Diff（差异）算法**’。

系统会自动对比‘前端提交的新文件列表’和‘数据库旧列表’，精准识别出哪些是**新增**（需要转码）、哪些是**删除**（需要清理）、哪些是**不变**。

这里比较考验的是**数据库事务与消息队列的一致性**，因为我需要在确保数据库落库成功后，才触发后续的转码任务。”

第三阶段：解决性能难题 —— 异步解耦与削峰)

“最后是转码阶段，这是最容易把服务器打挂的环节。

因为 FFmpeg 非常吃 CPU，我引入了 **RocketMQ** 将‘投稿’和‘转码’彻底解耦。

我限制了消费者的**线程并发度**，利用 MQ 的削峰填谷能力，保护后端服务不被流量洪峰击垮。

同时，由于一个视频包含多个分片文件，这里涉及到一个‘**多线程并发下的状态最终一致性**’问题。我设计了一个‘**漏斗模型**’，保证只有当所有分片都处理完时，才由最后一个线程去更新视频的主状态。”

视频预上传

断点续传是怎么实现的？如果不小心断网了，下次怎么知道从哪传？

“我是基于 **Redis** 来实现断点续传状态管理的。

1. 用户开始上传前，我会生成一个唯一的 uploadId，并在 Redis 里存一个对象，记录这个文件总共有多少片，以及**当前成功传到了第几片**（初始化为0），还记录了存储路径和文件总大小
2. 每当一个分片上传成功，我就会更新 Redis 里这个对象的 chunkIndex。
3. 如果用户断网了，下次上传前，前端可以先查一下这个 uploadId 的状态。比如 Redis 里记录到了第 5 片，那前端就从第 6 片开始传，不需要重头开始。”

问题二：如果我把第 5 片还没传，直接传第 6 片（乱序/跳着传），你的系统支持吗？

在当前版本中，为了保证文件合并的准确性和逻辑的简单性，我设计的是**严格的串行上传策略**。

我在后端代码里加了一个校验逻辑：**上传分片的序号必须紧接着上一个成功的序号**。

比如 Redis 记录当前传完了第 5 片，那下一个来的请求必须是第 6 片。如果是第 7 片先到了，我的系统会抛出异常，拒绝接收。这样能确保服务器上保存的分片绝对是连续的，合并时不会出现空洞。”

但我也考虑过优化方案：如果要支持高并发乱序上传，我会在 Redis 中使用 **Bitmap** 或 **Set** 集合来记录已上传的 chunkIndex。只要最后 Set 的大小等于总片数，就代表上传完成。这样就能支持前端多线程并发上传，最大化利用带宽。目前为了快速上线，我选择了串行方案。”

这么多分片传到服务器上，你是怎么管理的？怎么防止文件名冲突？

我是按照 **‘日期 / 用户ID_UploadId / 分片序号’** 的目录结构来存储的。

1. 在预上传阶段，我会生成一个全局唯一的 uploadId。
2. 我在临时目录下创建一个文件夹，名字包含 日期 和 uploadId_userId，这样既隔离了不同用户的文件，又避免了同一个用户上传不同文件时的冲突。
3. 文件夹里面的分片文件，我直接用 **分片序号 (0, 1, 2...)** 来命名。这样在最后合并文件时，我只需要按数字顺序读取文件流，就能组装成原始视频，不需要额外的映射表。”

问题四：如果用户传了一半不传了，这些垃圾文件怎么处理？

针对这个问题，我做了两层设计：

1. **元数据层面：**我在 Redis 存储上传状态时，设置了 **1 天的过期时间**。如果用户一天没操作，这个 Key 就自动消失了，上传任务失效。
2. **文件层面：**（这里稍微补充一点设计）因为磁盘文件不会自动删除，所以我设计了一个**定时任务 (Spring Task)**，每天凌晨去扫描临时目录。如果发现某个文件夹超过 24 小时没有被合并（或者 Redis 里找不到对应的 Key 了），就视为垃圾文件进行物理删除，释放磁盘空间。”

正式上传

问题一：用户点击“投稿”按钮后，你的后端具体做了哪些事情？（宏观流程）

你的代码逻辑（心中有数）：

1. 接收参数（标题、封面、JSON格式的文件列表）。
2. 保存/更新 VideoInfoPost（审核表）和 VideoInfoFilePost（文件表）。
3. 判断是新增还是修改。
4. 把新增的文件发给 RocketMQ 去转码。

你可以这样回答：

“当用户点击投稿时，后端主要执行了一个‘保存元数据 + 触发异步处理’的流程，具体分三步：

1. **元数据落库**：首先，我会接收前端传来的视频标题、简介、封面以及**文件列表（包含 uploadId）**。我会将这些信息保存到‘**视频审核表**’中。这里我设计了‘**审核表**’和‘**发布表**’隔离的架构，确保用户的修改不会直接影响线上正在被观看的视频。
2. **增量比对**：如果是修改稿件，我会对比‘新提交的文件列表’和‘数据库里已有的列表’，计算出哪些是**新增的分片**，哪些是**需要删除的分片**，避免对没变动的文件重复转码。
3. **异步驱动**：对于新增的视频文件，我会在事务提交成功后，向 **RocketMQ** 发送一条消息。消费者收到消息后，才会开始进行耗时的转码和切片处理。接口本身只做简单的数据库操作，实现了快速响应。”

问题二：如果用户已经发过一个视频，现在想修改（比如删掉第 2 集，补上第 3 集），你的系统怎么处理？

面试官意图：考察增删改查中复杂的“差异更新（Diff）”逻辑。

你的代码逻辑（心中有数）：

- 代码里做了 uploadFileMap 和 dbInfoFileList 的对比。
- delFileList：数据库有但前端没传的。
- addFileList：前端传了但没有 fileId 的。
- 只把 addFileList 发 MQ，把 delFileList 软删除并加到 Redis 待删队列。

“这是一个典型的**增量更新**场景。为了节省服务器资源，我不会把所有视频重新转码一遍，而是做了一个 **Diff（差异对比）** 逻辑：

1. **识别删除**：我会遍历数据库里的旧文件列表，如果发现某个文件在前端新提交的列表中**不存在了**，就把它标记为‘待删除’，并放入 Redis 队列，稍后异步清理物理文件。
2. **识别新增**：同时，我会检查前端提交的新列表，如果发现某个文件**没有关联的 fileId**（说明是新上传的），就把它标记为‘新增’。
3. **精准调度**：最后，我只将‘**新增**’的那部分文件发送给 RocketMQ 去触发转码。对于没变动的文件，直接保留原样。
4. **状态流转**：只要有新增文件，视频状态就会自动变为‘**转码中**’；如果只是删减或改标题，状态则直接流转为‘**待审核**’。”

问题一：用户投稿成功后，视频是怎么被处理的？（讲清楚核心流程）

你的回答逻辑：

“在用户接口返回成功后，真正的视频处理才刚刚开始。我设计了一个基于 **RocketMQ** 的异步处理流水线，主要包含四个步骤：

1. **异步解耦**：接口层只发消息，立即响应用户。后台的消费者服务从 MQ 拉取任务。
2. **分片合并**：消费者首先根据 UploadId，把服务器临时目录下的所有分片文件，按照序号顺序合并成一个完整的 MP4 源文件。
3. **转码切片**：接着调用 **FFmpeg** 引擎。先计算视频时长，然后将视频进行标准化转码（统一为 H.264 编码），并切割成 HLS (m3u8 + ts) 格式。这样做是为了适应不同带宽的播放需求，实现‘秒开’。
4. **清理收尾**：最后删除临时的分片文件和中间态的 MP4，只保留切片文件，并更新数据库状态。”

问题二：视频转码非常消耗 CPU，如果同时有 1000 个人投稿，你的服务器不就卡死了吗？（讲性能与削峰）

面试官意图：考察对计算密集型任务的资源隔离与调度。

你的回答逻辑：

“是的，这是一个典型的**计算密集型**场景。为了防止流量高峰把服务器 CPU 打满，我做了两层防护：

1. **MQ 削峰填谷**：我利用 RocketMQ 作为缓冲池。哪怕前端瞬间涌入 1000 个请求，消息也只是堆积在 MQ 里，不会直接冲击后台处理服务。
2. **消费者限流（关键点）**：我严格限制了转码消费者的**并发线程数**（在我的配置中只有 2 个线程）。这意味着，无论堆积了多少任务，我的服务器同一时刻只处理 2 个视频。虽然处理速度是恒定的，但保证了服务器的稳定性，不会因为 CPU 上下文切换过于频繁而宕机。”

一个视频可能包含多个文件（比如分集），你怎么判断整个视频全部处理完了，可以变成“待审核”状态？（讲状态一致性）

面试官意图：考察并发任务结束后的状态汇总（Funnel/漏斗模型）。

你的回答逻辑：

“这是一个多任务并发下的状态同步问题。我采用的是‘**最后完成者提交（漏斗模型）**’的策略：

1. **单个完结**：任何一个文件转码完成后（无论成功失败），都会先更新它自己的文件状态。
2. **全局检查**：紧接着，当前线程会去查询数据库，看该视频 ID 下是否还有状态为‘转码中’的文件。
3. **状态流转**：
 - 如果有其他文件还在跑，当前线程就只做自己的收尾工作。
 - 如果查询结果为 0（说明我是最后一个完成的），那么**当前线程**就有责任将整个视频的主状态更新为‘待审核’（或‘转码失败’）。

通过这种机制，我不需要引入额外的分布式锁或复杂的协调器，就保证了最终状态的一致性。”

问题四：转码过程中如果 FFmpeg 报错了，或者文件损坏了，系统怎么容错？（讲可靠性）

面试官意图：考察系统的健壮性。

你的回答逻辑：

“针对转码失败，我设计了三级容错机制：

1. **MQ 自动重试**：如果是网络抖动或临时 IO 占用导致的异常，我会抛出运行时异常，利用 RocketMQ 的重试机制（默认重试多次）来自动恢复。
2. **状态标记**：如果重试多次依然失败（说明文件可能损坏），我会捕获异常，将该文件的状态标记为‘**转码失败**’，并将整个视频的状态置为‘需要检查’。
3. **资源兜底**：无论成功还是失败，我都在 finally 代码块中强制执行资源清理逻辑，确保临时的分片文件和中间文件被删除，防止磁盘空间泄漏。”

在线人数

你的心跳机制具体是怎么实现的？

你的回答（基于代码 reportVideoPlayOnline）：

“我设计了一套‘**双 Key 保活机制**’。客户端每隔 5 秒轮询一次接口：

1. **去重 Key (user:fileId:deviceId)**：这是一个带有 8 秒过期时间的 Key。
 - 当请求到来时，我先检查这个 Key 是否存在。如果不存在，说明是**新用户**进入，我就对总人数 Key 执行 INCR 自增操作。
 - 如果存在，说明是**老用户**保活，我只给它续期 (expire)，不增加人数。
2. **计数 Key (count:fileId)**：存储当前的在线总人数。每次有新用户进入时自增，并设置一个稍长的过期时间作为兜底。

这样设计的好处是，完全利用 Redis 的原子性，避免了应用层复杂的锁竞争，性能非常高。”

用户直接关掉浏览器，你怎么知道他下线了？人数怎么减？

这是这个模块的一个设计难点。因为 HTTP 是无状态的，我无法像 WebSocket 那样立刻感知连接断开。

所以我利用了 **Redis 的过期事件通知机制**。

1. **原理**：当用户的‘去重 Key’因为没有收到心跳而自然过期（TTL归零）时，Redis 会发布一个 expired 事件。
2. **实现**：我在后端继承了 KeyExpirationEventMessageListener。一旦监听到某个用户的 Key 过期了，我就解析出其中的 filed，然后对总人数 Key 执行 DECR 自减操作。
这就是一种‘**被动触发**’的下线机制，实现了人数的自动回收。”

你的定时任务是分布式的吗？怎么防止多台机器重复写数据库？

你的回答（基于代码 statisticsData 逻辑）：

“因为我的服务是多实例部署的，如果直接用 @Scheduled，凌晨 0 点时所有机器都会去执行数据清洗，导致数据库数据重复或死锁。

所以我引入了 **Redisson 分布式锁**。

具体流程是：

1. **抢锁**：任务触发时，所有节点尝试去 Redis 抢一把名为 lock:daily_stat_job 的锁（设置 0 等待时间，抢不到立刻放弃）。
2. **执行**：只有**抢到锁**的那一台机器，才会执行‘从 Redis 拉取昨日数据 -> Stream 流聚合 -> 写入 MySQL’的 ETL 逻辑。
3. **释放**：任务结束后释放锁。
这样保证了整个集群中，**同一时刻只有一台机器**在做持久化操作。”

登录校验&AOP

你为什么要把登录校验拆成两部分？全在 Gateway 做不行吗？

这是一个基于**业务形态差异**的架构决策：

1. **审核端（后台）**：业务极其标准，所有接口都**强制需要管理员登录**。所以我在 **Gateway** 层实现了一个全局拦截器，直接解析 Token。如果没有权限，直接返回 401，根本不需要请求到达具体的微服务，这样能**节省服务器资源，保护内部服务**。
2. **用户端（前台）**：业务非常灵活。
 - 比如‘视频播放’接口，游客也能看，但登录用户能看高清；
 - 比如‘点赞’接口，必须登录。
 - 如果全在 Gateway 做一刀切的拦截，配置会非常复杂且难以维护。

- 所以我在应用层使用 **AOP 注解（如 @CheckLogin）**，标记在 Controller 方法上。这样哪个接口需要登录，加个注解就行，**颗粒度更细，更贴近业务。**”

消息通知的AOP实现的流程是什么

你的回答（严格基于代码逻辑）：

“我的实现逻辑主要基于 **@Around 环绕通知**，整个流程分为四个严格的步骤：

第一步：业务优先执行（Proceed）

我使用了 @Around 注解拦截带有 @RecordUserMessage 的方法。

进入切面后，我首先执行 joinPoint.proceed()。**这非常关键**，这意味着我先让 Controller 里的点赞/评论业务跑完。如果业务逻辑抛出异常（比如数据库挂了），代码就不会继续往下走，从而避免了‘业务失败却发了通知’的问题。拿到 ResponseVO 后，我暂时持有它。

第二步：基于反射的参数清洗

业务执行成功后，我开始利用 **Java 反射** 解析上下文。

我遍历了 method.getParameters()，通过字符串匹配的方式（如 if name.equals("videoid")），从参数列表中提取出 videoid、content 等关键信息。这基于我们团队的命名规范，约定了所有接口参数名必须统一。

第三步：业务类型适配（特殊逻辑）

这里有个特殊的业务场景：前端的‘点赞’和‘收藏’复用了同一个接口，只是 actionType 参数不同。

所以我在切面里做了一个判断：如果检测到 actionType 等于‘收藏类型’，我会强制覆盖注解原本配置的 MessageType，将其修正为 COLLECTION，确保通知类型准确。

第四步：用户信息获取与下发

最后，我通过 RequestContextHolder 拿到当前的 Request，从 Header 里取出 Token，然后**调用 Redis 组件**查询出当前操作人的信息（TokenInfoDto）。

数据都齐了之后，调用 userMessageService.saveUserMessage 完成消息的存储和发送，最后返回第一步拿到的 ResponseVO 给前端。”

登录校验的AOP是什么

我的用户端登录校验是一个基于 **@Before 前置通知** 的轻量级拦截机制，整个流程分为 **四个步骤**：

第一步：精准拦截（定义切点）

我定义了一个自定义注解 @GlobalIntercept。

只要 Controller 方法上标记了这个注解，AOP 就会在**方法执行之前**介入。通过反射获取方法签名，我首先检查注解中配置的 checkLogin 属性是否为 true。如果为 false，则直接放行，不做校验。

第二步：获取 HTTP 上下文

确定需要校验登录后，我利用 Spring 的 **RequestContextHolder** 获取到当前线程绑定的 `HttpServletRequest` 对象。

这是非 Controller 层获取请求的标准做法，它底层基于 `ThreadLocal`，保证了线程安全。

```
HttpServletRequest request =  
((ServletRequestAttributes)RequestContextHolder.getRequestAttributes()).getRequest();
```

第三步：提取凭证

接着，我从 Request 的 **Header** 中获取名为 token 的字段（常量 `TOKEN_WEB`）。

如果 Header 里连 Token 都没有，我直接抛出自定义异常 `BusinessException(CODE_901)`，由全局异常处理器拦截并返回‘未登录’错误码，**阻断**后续业务逻辑的执行。

第四步：Redis 验证与放行

拿到 Token 后，我将其作为 Key（拼接前缀），去 **Redis** 查询对应的 `TokenInfoDto`。

- 如果查不到（Token 过期或伪造），同样抛出异常进行阻断。
- 如果查到了，说明用户合法，**AOP 方法结束，自动放行**，请求继续进入 Controller 执行具体的业务逻辑。”

你的拦截工厂是怎么设计的以及你为什么要自定义

“之所以自定义拦截工厂，是因为官方的标准 Filter 无法满足我项目中‘**多渠道、差异化**’的鉴权需求。

在我的审核端（Admin）业务中，存在两种截然不同的请求场景：

1. **普通 API 请求**：Token 放在 **Header** 里，这是标准做法。
2. **文件上传/预览请求**：这里的 `rawPath` 包含 `file`。因为涉及到浏览器直接访问或某些上传组件的限制，Token 只能放在 **Cookie** 里传递，无法放在 Header。

• 第一步：白名单放行

我首先解析 `request.getURI().getRawPath()`。如果路径包含 `account`（比如登录接口），直接调用 `chain.filter(exchange)` 放行，不进行 Token 校验。

• 第二步：策略分发（关键点）

针对非白名单路径，我做了一个**策略判断**：

- 如果路径包含 `file`（文件相关），我从 **Request Cookies** 中提取 Token。
- 如果是其他普通路径，我从 **Request Headers** 中提取 Token。

• 第三步：断言与阻断

拿到 Token 后，我进行非空校验。如果 Token 为空，直接抛出 `BusinessException`。

这里特别说明一下，因为 Gateway 是基于 **WebFlux** 的，所以这里抛出的异常会被我的全局异常处理器（`GlobalErrorWebExceptionHandler`）捕获，最终返回标准的 JSON 格式错误码（401）给前端。”

```
public class AdminFilter extends AbstractGatewayFilterFactory {  
  
    @Override  
    public GatewayFilter apply(Object config) {  
        return ((exchange, chain) -> {
```

```

        ServerHttpRequest request = exchange.getRequest();
        String rawPath = request.getURI().getRawPath();
        if(rawPath.contains("account")){
            return chain.filter(exchange);
        }
        String token = getToken(request);
        if(rawPath.contains("file")){
            token = getTokenFromCookie(request);
        }
        if(token.isEmpty()){
            throw new BusinessException(ResponseCodeEnum.CODE_901);
        }
        return chain.filter(exchange);
    });
}

```

你这里抛出 BusinessException，Gateway 能捕获到吗？它不是 Servlet 容器哦。

面试官：“你的网关异常处理是怎么做的？为什么没用 @ControllerAdvice？”

你的回答：

(1. 指出架构差异 —— 关键得分点)

“因为 Spring Cloud Gateway 的底层是基于 **WebFlux (Netty)** 的，是**异步非阻塞**模型。而普通的 @ControllerAdvice + @ExceptionHandler 是基于 **Spring MVC (Servlet)** 的同步阻塞模型。在 Gateway 里，Servlet 的那套机制是不生效的。所以我必须实现 **WebExceptionHandler** 接口来接管异常。”

(2. 解释实现细节 —— 展现代码能力)

“我的实现逻辑主要分为三步：

1. **优先级控制：** 我加了 **@Order(-1)** 注解。这是为了让我的处理器优先级高于 Spring 默认的 DefaultErrorWebExceptionHandler，否则系统会默认返回一个 HTML 的报错页面，而不是我想要的 JSON。
2. **数据包装：** 因为 Netty 底层处理的是字节流，所以我不能直接返回对象。我必须把 JSON 字符串转换成字节数组，然后用 **DataBufferFactory** 包装成 **DataBuffer** 数据块。
3. **响应式写入：** 最后，我调用 response.writeWith(Mono.just(dataBuffer))，把这个数据块作为一个**异步任务 (Mono)** 写入响应流，从而完成非阻塞的异常返回。”

```

public class GatewayExceptionHandler implements WebExceptionHandler {
    private static final String STATUS_ERROR = "error";

    @Override
    public Mono<Void> handle(ServerWebExchange exchange, Throwable ex) {

```

```
//是 webFlux 的核心。在响应式编程里，你不能直接“返回结果”，而是要返回一个“任务（Publisher）”。
// `Mono<Void>` 的意思就是：**“这是一个异步任务，任务执行完不需要返回具体数据，只要告诉系统‘我做完了’就行”**
// （因为数据通过 Response 写入流里了）。
ServerHttpResponse serverHttpResponse = exchange.getResponse();
//告诉前端：“我要给你返回 JSON 格式的数据”。
serverHttpResponse.getHeaders().setContentType(MediaType.APPLICATION_JSON);
ResponseVO responseVO = getResponse(exchange,ex);
//以前在 Servlet 里，我们直接 `response.getWriter().write(String)`。
//但在 Gateway (Netty) 底层，数据不是字符串，而是**字节流**。
//wrap(...)`**：把你的 JSON 字节数组（`byte[]`），**包装**成一个 Netty 能看懂的**数据块**（叫 `DataBuffer`）
DataBuffer dataBuffer =
serverHttpResponse.bufferFactory().wrap(JsonUtils.converObj2Json(responseVO).getBytes(StandardCharsets.UTF_8));
return serverHttpResponse.writeWith(Mono.just(dataBuffer));
}
```